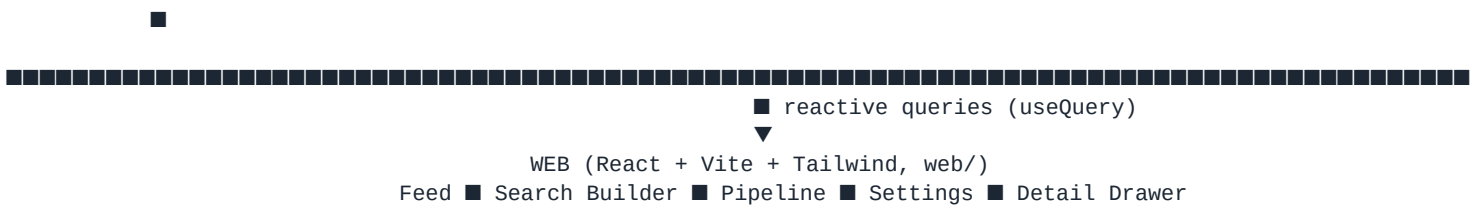
CarHunter — System Architecture

CarHunter documentation · source: ARCHITECTURE.md · generated June 2026

CarHunter — System Architecture

Status: M0 deliverable. Every later milestone must conform to this document. **Scope override (standing, user-issued 2026-06-12):** KSL-only build. Facebook Marketplace is deferred — the design below stays multi-source, but no FB credentials exist and no gate runs against FB. Where the spec says "both sources," read "KSL now, FB pluggable later."

1. System overview



Separation of concerns:

- **Scrapers (Python, `scrapers/`)** fetch + normalize only. No scoring, no dedupe, no DB access. They emit the §5 normalized listing shape and POST it. They run **only** inside Daytona sandboxes (RULES #6), torn down after each run.
- **Convex (`convex/`)** owns all state and all business math: dedupe, price history, status lifecycle, valuation, recon, scoring, alert dedupe, cron scheduling.
- **Web (`web/`)** renders state and issues decisions (Pursue/Pass/Contacted, pipeline moves, settings). Zero business math in the client — it displays fields the backend computed.

2. Repository layout

```
/
├── ARCHITECTURE.md            ← this file
├── README.md                  ← M9 deliverable (§10)
├── package.json               ← root: convex dep + npm workspace ["web"]
├── convex/
│   ├── schema.ts              ← §3 schema + partsCosts + scoring-provenance fields (§5
below)
│   ├── searches.ts            ← buy-box CRUD, listActive, markRun
│   ├── listings.ts            ← upsertFromScrape, feed, setDecision, markStale
│   ├── scoring.ts             ← scoreListing (internal), §4 engine
│   ├── comps.ts               ← getOrFetchComp action, 7-day cache, upsertComp (operator/
MCP path)
│   ├── partsCosts.ts          ← median lookup by year|make|model|part, seed mutation
│   ├── pipeline.ts            ← stage moves, targetBuy/walkAway, notes
│   ├── alerts.ts              ← sendHotAlert action + dedupe rule
│   ├── crons.ts               ← per-minute due-search dispatch; daily stale-mark + comp
refresh
│   ├── daytona.ts             ← runSearchInSandbox action (create → exec → teardown)
│   ├── http.ts                ← POST /ingest (shared secret)
│   ├── seed.ts                ← settings row, 7 §2 buy-box searches, partsCosts from CSV
│   └── lib/
│       ├── dedupe.ts          ← §5 dedupe key (single source of truth — server-side only)
│       ├── scoreMath.ts       ← pure §4 formula helpers (unit-tested)
│       ├── reconRules.ts      ← keyword classifier + parts-based recon math (unit-tested)
│       ├── depreciationCurve.ts ← flagged fallback valuation
│       └── marketcheck.ts     ← comp-source client (REST driver if key; otherwise cache-
only)
├── scrapers/
│   ├── run.py                 ← sandbox entrypoint: config → scraper(s) → batch POST /
ingest
│   ├── ksl.py                 ← KSL JSON API client (from KSLHax hax.py, refactored)
│   ├── facebook.py            ← DEFERRED stub: exits with explicit "FB disabled" error
│   ├── parse.py               ← shared normalizer + private-party hard filter
│   ├── car_brands.py          ← make/model dictionary (generated from reference car-brands.
js)
│   ├── requirements.txt
│   ├── tests/
│       ├── fixtures/          ← saved KSL API JSON + (future) FB HTML
│       └── test_parse.py, test_normalized_shape.py
└── web/
```

```
■   ■■■ package.json, vite.config.ts, tailwind.config.js, index.html
■   ■■■ src/
■       ■■■ main.tsx, App.tsx  ← ConvexProvider + view router (tab nav, mobile bottom bar)
■       ■■■ views/             ← FeedView, PipelineView, SearchBuilderView, SettingsView
■       ■■■ components/        ← DealCard, ScoreChip, HotRibbon, SourceBadge, FilterBar,
■       ■                        DetailDrawer, PriceHistorySparkline, CompPanel,
ReconMathTable,
■       ■                        KanbanBoard, KanbanColumn, PipelineCard, EmptyState,
ErrorState,
■       ■                        Skeleton, ConfirmButton
■       ■■■ lib/               ← formatters (money, miles, relative time), useFeedFilters
■■■ data/carpart_prices.csv    ← ~6,963 Grade-A used engine/trans price rows (seeds
partsCosts)
■■■ reference/                 ← the three provided codebases (read-only, never imported at
runtime)
```

3. Data flow (scrape → ingest → score → feed → alert)

1. **Dispatch.** `crons.ts` runs every minute: `searches.listDue` returns active searches where `lastRunAt + intervalMinutes*60000 <= now` (or `lastRunAt` unset). For each, schedule `daytona.runSearchInSandbox` — independent invocations, concurrency-capped (3), each wrapped in try/catch that writes `search.lastError` and never blocks siblings (M6 gate).
2. **Scrape.** The sandbox receives one JSON config (search fields + ingest URL + secret — the FB session cookie would ride here too; never a password, RULES #5). `run.py` picks scrapers by `sources`, runs each with retry/backoff, normalizes via `parse.py`, drops dealer listings (hard filter, RULES #4), and POSTs batches of ≤50 to `/ingest`. Exit code ≠ 0 on total failure.
3. **Ingest.** `http.ts` checks `X-Ingest-Secret` against the `INGEST_SECRET` env var, validates shape, and calls `listings.upsertFromScrape`. Per listing:
4. compute `dedupeKey` **server-side** (one implementation, no cross-language drift): `vin` if present, else `sha1(year|make|model|round(mileage,-3)|zip3)` (§5; mileage rounded to nearest 1,000 half-up; zip3 = first 3 digits of zip).
5. new key → insert, `status:"active"`, `priceHistory:[{price, at}]`, schedule scoring.
6. seen key, lower price → append `priceHistory`, set `status:"price_drop"`, reschedule scoring (re-alert becomes possible).
7. seen key, same/higher price → update `lastSeenAt`/`daysListed` only.
8. **Score.** `scoring.scoreListing` (§6 of this doc) writes `estValue/estRecon/estFees/ estProfit/dealScore/hot` + provenance (`compSource`, `compSampleSize`, `reconSource`, `reconBreakdown`, `scoreBreakdown`).
9. **Feed.** `listings.feed` query — ranked by `dealScore` desc via index, filterable (source, make, price, min profit, min score, max days listed, status). Convex reactivity pushes updates; no client polling.
10. **Alert.** Scoring flips `hot` → schedules `alerts.sendHotAlert`. Dedupe rule: a listing alerts once; it may alert again only if current price < price at last alert. Channel = email (Resend) / SMS (Twilio) when keys exist; otherwise an `alerts` row with `channel:"log"` still records the event (visible in-app) so the dedupe logic is real and testable without third-party keys.
11. **Lifecycle.** Daily cron: listings with `lastSeenAt` older than 48h → `status:"gone"`; comps older than 7 days that still back active listings → refresh attempt.

4. API surface

HTTP (Daytona sandbox → Convex):

Route	Auth	Body	Behavior
`POST /ingest`	`X-Ingest-Secret` header == `INGEST_SECRET` env	`{ searchId, source, listings: NormalizedListing[] }`	Validates §5 shape, upserts batch, returns `{inserted, updated, priceDrops, skipped}`

Convex functions (client-callable unless marked internal):

Module	Function	Kind	Purpose
searches	`list`, `get`	query	builder views
searches	`create`, `update`, `toggleActive`, `remove`	mutation	buy-box CRUD
searches	`listDue`	internal query	cron dispatch
searches	`markRun(searchId, {error?, newDeals?})`	internal mutation	lastRunAt/lastError/newDealsLastRun
listings	`feed(filters, paginationOpts)`	query	ranked deal feed
listings	`get(id)`	query	detail drawer
listings	`setDecision(id, "pursue"\	"pass"\	"contacted")`
listings	`upsertFromScrape(batch)`	internal mutation	§3 dedupe/price-history/status
listings	`markStale`	internal mutation	48h → gone
scoring	`scoreListing(listingId)`	internal action	full §4 pass
comps	`getOrFetchComp(ymm, mileageBucket)`	internal action	cache → MarketCheck → curve
comps	`upsertComp(row)`	internal mutation	cache write (also the operator/MCP seeding path)
partsCosts	`lookup(year, make, model, part)`	query	median + sampleSize (drawer shows its work); exact year first, then nearest year within ±2 (`matchedYear` reports which); null when the YMM has no data
partsCosts	`seedBatch(rows)`	internal mutation	CSV import
pipeline	`board`	query	kanban, grouped by stage
pipeline	`moveStage`, `setNumbers`, `setNotes`	mutation	board ops
alerts	`sendHotAlert(listingId)`	internal action	channel send + alert row
settings	`get`, `update`	query/mutation	single row

5. Schema (§3 verbatim + documented additions)

Spec §3 tables are adopted as written: `searches`, `listings`, `comps`, `pipeline`, `alerts`, `settings`. Additions (each justified, none alters a §3 field or spec number):

- `partsCosts` (mandated by M1): one aggregated row per `key = "year|make|model|part"` (part ∈ "Engine" | "Transmission"), fields: `year, make, model, part, key, medianPrice, sampleSize, variants: [{variant, medianPrice, numListings}], refreshedAt`. Aggregation: the median over all per-variant price observations in the CSV for that YMM+part; `sampleSize` = total listings. Index `by_key`.
- `listings` **provenance fields** (all optional; required by LOOP*PROMPT valuation/recon sections — "comp + recon source logged on every listing", "drawer shows the recon math"):* `compSource`

*("marketcheck*sold"|"marketcheck_active"|"cache"|"curve"), `compSampleSize`, `compRef` (comps row id), `reconSource` ("parts"|"keyword"|"base"), `reconBreakdown: [{label, amount, meta?}]`, `scoreBreakdown: {profit, marginPct, freshness, mileageFit, titleBonus}`, `lastAlertPrice` (alert-dedupe anchor). Extra index: `by_hot` (alert/ops queries), `by_lastSeenAt` (stale sweep).

- `searches.lastError`, `searches.newDealsLastRun` (optional) — §8 search builder shows last-run time and # new deals; M6 gate requires failure isolation to be observable.
- `settings.marketcheckKey` (optional) — §8 Settings view lists a MarketCheck key field.
- `settings.alertChannelFallback` ("log") — implicit; not a schema field, a behavior.
- `comps.source` (optional) — comp-set provenance ("marketcheck*sold*"|"*marketcheck*active"); mirrors the listing's `compSource` so cached comps stay auditable.
- `alerts.by_listing`, `pipeline.by_listing` **indexes** — alert dedupe and pursue-twice lookups are by listing id; without these they would be table scans.

6. Valuation + recon (the scoring engine, §4 + standing overrides)

Step 1 — estValue. Comp chain, in order:

1. `comps` cache by `ymm` + `mileageBucket` (20k-wide buckets: "60k-80k"), fresh = `refreshedAt` within **7 days**.
2. **MarketCheck** (RULES #3a: always attempted before any curve):
3. VIN present → neoVIN decode + price prediction w/ comparables; car history detects relists/prior drops (feeds `daysListed`/`priceHistory`).
4. No VIN → active-car search (clean title, same YMM, mileage bucket, near 84104) for asking comps, cross-checked against last-90-days **sold** comps — *sold beats asking when both exist* (`compSource:"marketcheck_sold"` > `"marketcheck_active"`).
5. Transport: `lib/marketcheck.ts` client. With `MARKETCHECK_KEY`/`settings.marketcheckKey` → REST driver, fully unattended. Without a key (this build): the harness operator uses the **MarketCheck MCP tools** to fetch comps and writes them through `comps.upsertComp` — same table, same TTL, `compSource` reflects the MCP query type. The app code path is identical either way: it reads the cache and only knows comp provenance.
6. Every MarketCheck result is cached into `comps` (7-day TTL) with sample size.
7. **Depreciation curve fallback** (`lib/depreciationCurve.ts`): only when cache is empty/stale and no MarketCheck path succeeded. Flagged `compSource:"curve"` → amber in UI, and **curve-valued cars never trigger HOT alerts** (RULES #3a).

Adjustments on the comp anchor: mileage delta vs bucket median ±$0.06/mile; AWD/4WD +3%; **salvage/rebuilt title:** `estValue = 0.70 × clean-title comp value` (standing override, replaces §4's ×0.65; comps must be pulled from clean-title vehicles only — never anchor on other salvage listings).

Step 2 — estRecon (`lib/reconRules.ts`). Base **$400** always. Then classify description+title text:

1. Engine failure keywords ("blown engine", "needs engine", "knocking", "no compression", "won't start"+engine context) → `medianPrice(YMM, Engine) + $1,300` labor.
2. Transmission keywords ("bad trans", "needs transmission", "slipping", "won't shift") → `medianPrice(YMM, Transmission) + $900` labor.
3. Generic broken ("mechanic special", "doesn't run", "as-is, broken", no component named) → the **pricier** of the YMM's engine/trans medians + its labor (worst case protects margin).
4. YMM not in `partsCosts` → §4 flat keyword bumps, `reconSource:"keyword"` (amber in UI).

5 Non-drivetrain issues → §4 bumps as written: salvage/rebuilt +$2,500, doesn't-start family +$2,000 (only when not already priced via parts), needs work/as-is +$1,000, accident/damage +$800, check engine +$600, new tires/brakes –$200. `reconBreakdown` records every line (part, median used price, sample size, labor, bump) so the drawer shows the math (RULES #3b).

Step 3 — estFees. `settings.feesFlat`, default **$400**.

Step 4 — profit + score (`lib/scoreMath.ts`, pure + unit-tested):

```
estProfit = estValue − price − estRecon − estFees
dealScore = 45·clamp(estProfit/4000, 0, 1)
          + 20·clamp((estValue − price)/estValue, 0, 1)
          + 15·freshnessBonus(daysListed)        // 1 − clamp(daysListed/30, 0, 1)
          + 10·mileageFit(mileage, buyboxBand)   // 1 inside band; linear falloff to 0 at
±30k outside
          + 10·titleBonus(titleStatus)           // clean 1.0 · unknown 0.4 · rebuilt 0.2 ·
salvage 0.1
hot = estProfit ≥ settings.marginThreshold (1500)  AND compSource ≠ "curve"
```

(The freshness/mileageFit/titleBonus shapes are architecture decisions — the spec fixes the weights, not the inner curves; clean title = full 10 as specified.)

Step 5 — suggested numbers (written to pipeline on Pursue):

```
targetBuy = estValue − estRecon − estFees − marginThreshold
walkAway  = estValue − estRecon − estFees − marginThreshold·0.6
```

7. Scrapers (KSL engine; FB deferred)

`ksl.py` (from `KSLHax-1.2/hax.py`, behavior preserved, quality raised):

- Build the search-path segments from the config using the reference grammar: `make/{A;B}/model/{X;Y}/mileageFrom/N/mileageTo/N/priceFrom/N/priceTo/N/zip/Z/miles/R/titleType/Clean+Title/yearFrom/Y/yearTo/Y/sellerType/For+Sale+By+Owner`.
- POST `https://cars.ksl.com/nextjs-api/proxy?` with the reference's exact envelope (`endpoint:"/classifieds/cars/search/searchByUrlParams"`, body = segments + `["perPage",24,"page",N,"es_query_group",null]`, Host/Origin/UA headers).
- Paginate until empty page or `max_pages`; retry/backoff (3 attempts, waits 2s·4s) on every network call; structured JSON logs to stderr; no bare asserts (replaced with typed errors). No Referer header — the reference computes one but never sends it (hax.py posts the original dict); wire fidelity wins.
- Field map = reference `data_types/car.py`: `makeYear`→`year`, `titleType`→`titleStatus`, `sellerType` `"Dealership"`→`dealer` (dropped), `"For Sale By Owner"`→`private`, photo id → photo URL, `createTime/displayTime`→`daysListed`, `id`→`sourceListingId`.

`parse.py` — shared normalizer: KSL dict → §5 normalized JSON; title-based year/make/model/ trim extraction backed by `car_brands.py` (generated from reference `car-brands.js`) for free-text fields; mileage text parsing ("87K"→87000, reference `search.py` logic); title-status keyword detection in descriptions; **dealer hard-filter** (sellerType + dealer-phrase heuristics). Pure functions, fixture-tested (M2 gate).

`facebook.py` — deferred stub that raises `ScraperDisabled("facebook: deferred by user override 2026-06-12")`. `run.py` logs and skips disabled sources without failing the run (failure isolation). The FB reuse plan (cookie injection from settings, abort_requests image-blocking, scroll/parse loop from the two FB references) is recorded in PROGRESS.md.

`run.py` — entrypoint: `--config '<json>'`; for each enabled source: scrape → normalize → filter → POST `/ingest` in batches with retry/backoff; exits non-zero only if **all** sources failed; always emits a final JSON summary line for the

Daytona action to parse.

8. Daytona integration (`convex/daytona.ts`)

`runSearchInSandbox(searchId)` action:

1. Load search + settings + `INGEST_SECRET`; serialize the sandbox config.
2. Create sandbox via the Daytona REST API (`DAYTONA_API_KEY` env; raw REST, no SDK dependency): prebuilt `carhunter-scraper` snapshot with Python 3.11 + Playwright + `scrapers/` baked in (snapshot build is a first-deploy prerequisite — README §10 item; it cannot be built from this egress-blocked container). KSL path is requests-only — Playwright rides along for the FB re-enable.
3. Exec `python run.py --config <json>` with a hard timeout (120s); stream logs.
4. `finally`: tear the sandbox down (RULES #6 — nothing persists), then `searches.markRun` with error or new-deal count.
5. Cron dispatch runs each search independently; one bad config writes `lastError` on its own search row and never blocks the others (M6 gate tests exactly this).

9. Dashboard (`web/`, §8)

Mobile-first (worked from the lot), Tailwind, Convex `useQuery`/`useMutation` (reactive — no polling). Every view has explicit loading (skeletons), empty, and error states. Visual language: green = money, red = pass/risk, amber = verify (curve comps, keyword recon, stale session). Keyboard-navigable; labeled controls; AA contrast.

1. **FeedView** — ranked `DealCard`s (photo, YMM+trim, miles, asking price large, estValue, estProfit green, `ScoreChip` color-tiered, `SourceBadge`, distance, days listed, View listing link, Pursue/Pass/Contacted). Red `HotRibbon` when hot; amber border when `compSource:"curve"` or `reconSource:"keyword"`. `FilterBar`: source, make, price, min profit, min score, max days listed, sort.
2. **PipelineView** — `KanbanBoard` Lead → Contacted → Negotiating → Bought → Flipped → Dead; drag between stages (keyboard fallback: move-stage menu); cards show targetBuy/walkAway + notes.
3. **SearchBuilderView** — create/edit all §3 search fields; active toggle; last run + # new deals + lastError surfaced.
4. **SettingsView** — margin threshold, flat fees, alert email/phone, FB session cookie paste (kept for re-enable; "session expired" banner slot), Daytona key, MarketCheck key.
5. **DetailDrawer** — photos, description, `PriceHistorySparkline`, `CompPanel` (comp source, sample size, median, freshness), `ReconMathTable` (line-by-line: part, median used price, sample size, labor — RULES #3b), profit math broken out, suggested target/walk-away, Pursue/Pass/Contacted.

Component contracts documented in each file (props, states, usage snippet) per LOOP_PROMPT M7.

10. Caching strategy

- **comps:** 7-day TTL (`refreshedAt`), keyed `ymm|mileageBucket`; daily cron refresh for buckets backing active listings; stale curve-valued listings rescore when a real comp lands.
- **partsCosts:** static dataset, seeded at M1; re-seed is an explicit operator action.
- **Convex reactive queries** are the client cache — the feed re-renders only changed rows (M9 verifies no unaffected-card re-renders; `DealCard` memoized by listing id + score).
- **Scrape dedupe:** server-side upsert by `dedupeKey` makes repeated scrapes idempotent (M4 gate: same batch twice → one row per key).

11. Error handling & operational rules

- Every network call (scraper HTTP, MarketCheck, Daytona SDK, Resend/Twilio) gets retry with exponential backoff (3 tries) and structured failure logs; no silent catches.
- No secrets in code or repo: `DAYTONA_API_KEY`, `INGEST_SECRET`, optional `MARKETCHECK_KEY`, `RESEND_KEY`, `TWILIO_*` live in Convex env vars / `.env.local` (gitignored). FB cookie — when re-enabled — lives in `settings.fbSessionCookie` (RULES #5), pasted via Settings UI.
- Scrapers never run on the app server (RULES #6); the only inbound path is `/ingest` with the shared secret.
- The app finds and ranks; it never contacts sellers or moves money (RULES #7).